```
//top_moor_fsm.v

`timescale 1ns / 1ps
module top_moor_fsm(
    input clk,
    input reset,
    input btnU,
    input btnD,
    output led
);

    wire clean_u, clean_d;

    debouncer u_deb_u (
        .clk(clk),
        .reset(reset),
        .noisy_btn(btnU),
        .clean_btn(clean_u)
    );

    debouncer u_deb_d (
        .clk(clk),
        .reset(reset),
        .noisy_btn(btnD),
        .clean_btn(clean_d)
    );

    reg prev_u =0, prev_d =0;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            prev_u <=0;
            prev_d <=0;
        end
        else begin
            prev_u <=clean_u;
            prev_d <=clean_d;
        end
    end
```

```
    wire trig_u =clean_u &&!prev_u;
    wire trig_d =clean_d &&!prev_d;

    wire fsm_clk =trig_u ||trig_d;

    reg x_val =0;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            x_val <=0;
        end
        else if (trig_u) begin
            x_val <=1'b1;
        end
        else if (trig_d) begin
            x_val <=1'b0;
        end
    end

    wire fsm_y;

    moor_fsm_101 u_moor_fsm_101 (
        .clk(fsm_clk),
        .reset(reset),
        .x(x_val),
        .y(fsm_y)
    );
    assign led =fsm_y;
endmodule
```

```
//top_mealy_fsm.v

`timescale 1ns / 1ps
module top_mealy_fsm(
    input clk,
    input reset,
    input btnU,
    input btnD,
    output led
);

    wire clean_u, clean_d;

    debouncer u_deb_u (
        .clk(clk),
        .reset(reset),
        .noisy_btn(btnU),
        .clean_btn(clean_u)
    );

    debouncer u_deb_d (
        .clk(clk),
        .reset(reset),
        .noisy_btn(btnD),
        .clean_btn(clean_d)
    );

    reg prev_u =0, prev_d =0;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            prev_u <=0;
            prev_d <=0;
        end
        else begin
            prev_u <=clean_u;
            prev_d <=clean_d;
        end
    end
```

```
    wire trig_u =clean_u &&!prev_u;
    wire trig_d =clean_d &&!prev_d;

    wire fsm_clk =trig_u ||trig_d;

    reg x_val =0;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            x_val <=0;
        end
        else if (trig_u) begin
            x_val <=1'b1;
        end
        else if (trig_d) begin
            x_val <=1'b0;
        end
    end

    wire fsm_y;

    mealy_fsm_101 u_mealy_fsm (
        .clk(fsm_clk),
        .reset(reset),
        .x(x_val),
        .y(fsm_y)
    );

    reg led_reg =0;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            led_reg <=0;
        end
        else begin
            if (fsm_y)        led_reg <=1'b1;
            else if (fsm_clk) led_reg <=1'b0;
        end
    end
    assign led =led_reg;
endmodule
```